FOR COMPLETE BEGINNERS

Stop copying the class.

Complete Modern Java, episode 26: inheritance, in Java and TypeScript

> One class building on another: reuse the parent's methods, add your own, and override one.

Is every fresh product a product? Yes. Is every product a fresh product? No. That is the test for inheritance.

Use it when one kind of thing really is a more specific kind of another, not just to reuse code.

In this episode

- is-a
- extends
- super(...) in a constructor
- Inherited methods
- @Override and super.describe()
- One list, its own version
- TypeScript override

Coming next

- Composition or inheritance (episode 27)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

milk.getName(), getPriceCents(), getDaysLeft()	`Milk, 99, 3`
@Override on a misspelt describe	`compile error`
milk.describe()	`Milk: 99, 3 days left`
describe() over a shelf of Product	`Rice: 199; Milk: 99, 3 days left`
shelf.ts	`the same two lines`

Every row was run, not predicted.

02 extends and super

super(name, priceCents) runs Product's constructor, so the price check still runs. Product has no constructor without arguments, so the child must call one.

The milk object holds a name and a price; they are private to Product, so FreshProduct uses Product's methods.

FreshProduct.java

```
class FreshProduct extends Product {
    private int daysLeft;

    FreshProduct(String name, int priceCents, int daysLeft) {
        super(name, priceCents);
        this.daysLeft = daysLeft;
    }

    int getDaysLeft() {
        return daysLeft;
    }

    @Override
    String describe() {
        return super.describe() + ", " + daysLeft + " days left";
    }
}
```

> milk.getName(), getPriceCents() and getDaysLeft() printed Milk, 99, 3; describe() printed Milk: 99, 3 days left.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 One shelf, its own version

Both are treated as Product, but milk really is a FreshProduct, so its own describe runs. That is polymorphism.

Shelf.java

```
ArrayList<Product> shelf = new ArrayList<>();
shelf.add(new Product("Rice", 199));
shelf.add(milk);
for (Product product : shelf) {
    IO.println(product.describe());
}
// Rice: 199
// Milk: 99, 3 days left
```

> A misspelt describe under @Override: method does not override or implement a method from a supertype.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript extends

The override keyword is TypeScript's version of the annotation. super must come before this in the constructor.

shelf.ts

```
class FreshProduct extends Product {
    daysLeft: number;
    constructor(name: string, priceCents: number, daysLeft: number) {
        super(name, priceCents);
        this.daysLeft = daysLeft;
    }
    override describe(): string {
        return super.describe() + ", " + this.daysLeft + " days left";
    }
}
```

> tsc quiet; the same two lines. this before super: TS17009.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Inheritance, in one look

1. Is every child really a kind of the parent? Then extends.
2. Call super(...) first in the constructor.
3. Type @Override (Java) or override (TypeScript).
4. super.method() reuses the parent instead of copying it.
5. Challenge: cheese at 350, 5 days left. Cheese: 350, 5 days left.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.